CS6666 COURSE PROJECT REPORT

CricketVerse

Author: Tejaswini Kunkulagunta

A decentralized Web3 cricket platform: dynamic NFT player cards, anti-scalping smart ticketing, escrowed fantasy tournaments, and zero-knowledge squad commitments

CONTENTS

1 Introduction 3
2 System Architecture 3
3 Smart Contract Layer 5
4 Core Mechanisms 7
5 Fantasy Engine & Zero-Knowledge Squad Commitments 8
6 Data Pipeline & Oracle 9
7 Frontend & User Experience 9
8 Testing, Security & Gas Analysis 10
9 Deployment & Operation 11
10 Limitations & Future Work 12
11 Conclusion 13
References 13

Abstract

CricketVerse is a decentralized application that rebuilds four parts of the cricket fan economy on a blockchain: collectibles, stadium ticketing, fantasy contests, and fan engagement. Eight Solidity 0.8.24 smart contracts implement an ERC-20 utility token ($CRIC), dynamic ERC-1155 player cards whose artwork is generated on-chain, ERC-721 match tickets with a hard-coded 15% resale ceiling and one-time turnstile redemption, an escrow marketplace, a tournament prize pool with a 95/5 payout split and rain-out refunds, an owner-authorized oracle relayer, and wallet-signed fan polls. A React 19 and TypeScript front end connects to the contracts through an EIP-1193 wallet and adds a browser-side zero-knowledge-style squad commitment scheme for fantasy lineups.

This report describes the architecture, contract design, core mechanisms, test evidence, and operating procedure of the project, based on the repository documentation. It also states clearly what is and is not enforced on-chain: the project runs against a local Hardhat chain, its 47 automated tests are not an independent audit, and several features are prototypes of a larger design.

1 Introduction

1.1 Background and motivation

Cricket is a large and passionate market, estimated in the project documentation at about $12.8 billion across broadcasting, memorabilia, fantasy sports, and stadium ticketing. Most of that value is held on centralized Web2 platforms, where users do not truly own what they buy and cannot verify how it is priced, scored, or paid out. The project targets four concrete weaknesses:

1. **Secondary-market scalping.** Bots drain primary ticket inventory within seconds and relist passes at 300% to 500% premiums, often with cloned barcodes. Organizers earn nothing from the markup.
2. **Strategy front-running in fantasy sports.** If lineups are public before a match locks, rivals can copy top strategists or counter declared squads. If they are hidden off-chain, operators can alter them.
3. **Static digital collectibles.** Conventional NFTs point to inert images that never reflect a player's real performance.
4. **Opaque prize custody.** Contest operators hold entry fees, which creates counterparty and insolvency risk.

1.2 Objectives

1. Enforce a maximum resale price for tickets in contract code, so the rule cannot be bypassed from the user interface.
2. Make ticket entry a single, permanent on-chain state change so a duplicated QR code cannot admit a second person.
3. Represent player cards as dynamic assets whose statistics and on-chain SVG artwork change with performance data.
4. Hold marketplace and tournament funds in non-custodial escrow with atomic settlement and refund guarantees.
5. Let fans commit to a fantasy squad without revealing it, using a hash commitment backed by a constraint-checking proof.

1.3 Scope and method

The report covers the full repository: contracts, scripts, tests, front end, and deployment tooling. Its content is drawn from the repository's README, System Requirements Specification, security review, and setup guide.

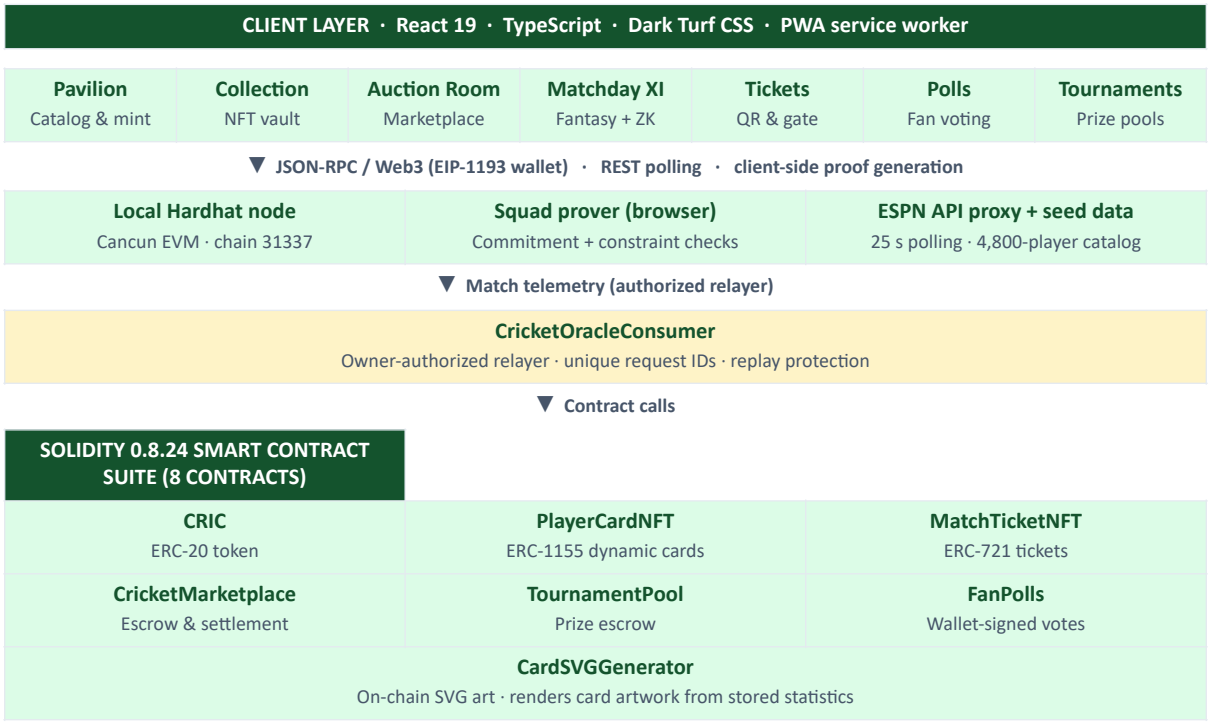
> **Reading note.** CricketVerse is a course prototype. Statements about enforcement refer to the local Hardhat deployment.

2 System Architecture

2.1 Hybrid design

CricketVerse is a hybrid system. Ownership, money, and rules that must be trustless (token balances, card and ticket ownership, escrow, resale limits, ticket redemption, tournament payouts, poll votes) live in smart contracts. High-volume or presentation work (the player catalog, live score polling, the six-ball simulator, fantasy point calculation, QR rendering, and squad proof generation) runs in the browser or a lightweight server layer. Figure 2.1 shows the layers.

Figure 2.1 – Layered architecture of CricketVerse



2.2 Technology stack

Layer	Technology	Role
Contracts	Solidity 0.8.24, OpenZeppelin, Hardhat	ERC-20/721/1155 assets, AccessControl, Ownable, ReentrancyGuard; `viaIR` with 200 optimizer runs.
Front end	React 19, TypeScript, Vite, Ethers.js	Single-page app; one context provider (`CricketContext.tsx`) owns wallet, chain, and sync state.
Wallet	MetaMask / any EIP-1193 wallet	Signs every feature transaction; gatekeeper wallets hold the scanner role.
Data	`seedData.ts`, `players_2000_2026.json`, ESPN API	Offline fallback, 4,800-player catalog, live score polling every 25 seconds.
Offline / PWA	`sw.js`, `manifest.json`	Service-worker caching and an installable app for gate verification on phones.

2.3 What is on-chain and what is not

Concern	Where it lives
CRIC balances; card ownership and minting	**On-chain** – contract state
Marketplace escrow and settlement	**On-chain** – `CricketMarketplace`
Ticket ownership, resale limit, redemption	**On-chain** – `MatchTicketNFT`
Tournament entries, refunds, prizes	**On-chain** – `TournamentPool` (events such as `PrizePaid`)
Fan poll votes	**On-chain** – `FanPolls`
Player catalog, six-ball over simulation	Server-backed demo data
Fantasy point calculation, squad proof generation, QR rendering	Browser (`scoringEngine.ts`)
Tournament winners	Entered by the organizer wallet

2.4 Development-server resilience

The Vite configuration includes a self-healing plugin for the local chain. As described in the project notes, it detects when the Hardhat node on port 8545 is unavailable, can restart it and redeploy the ecosystem, and answers early RPC calls with a valid warming-up response instead of a proxy error. This removes a common friction point in demos, where an idle node would otherwise leave the interface stuck.

3 Smart Contract Layer

3.1 Contract registry

All eight contracts are deployed by one script (`deploy_full_ecosystem.js`). The addresses below are the deterministic local Hardhat addresses recorded in the repository (shortened here).

Contract	Standard	Local address	Responsibility
`CRIC`	ERC-20, Burnable	`0x5FbD…0aa3`	Utility token; test faucet; capped supply.
`PlayerCardNFT`	ERC-1155	`0xe7f1…0512`	Dynamic cards with rarity tiers and hard supply caps.
`MatchTicketNFT`	ERC-721	`0x9fE4…a6e0`	Tickets, 15% resale ceiling, turnstile redemption.
`CricketMarketplace`	Escrow	`0xa513…C853`	Non-custodial exchange, atomic settlement, royalty, scalping check.
`CardSVGGenerator`	On-chain SVG	`0x2279…eBe6`	Renders base64 vector card art with milestone badges.
`CricketOracleConsumer`	Oracle relay	`0x8A79…C318`	Owner-authorized telemetry updates to cards.
`TournamentPool`	Escrow pool	`0xA51c…91C0`	Fantasy contest entry, escrow, 95/5 payout, refunds.
`FanPolls`	Polls	`0x0DCd…CD82`	One vote per address per poll; immutable counts.

3.2 CRIC utility token

`CRIC.sol` extends ERC-20 with burn support and owner control. The maximum supply is 100,000,000 CRIC, and 10,000,000 CRIC are minted to the deployer at genesis to fund liquidity and drop pools. An on-chain faucet lets each wallet claim test tokens (50,000 CRIC in the current demo deployment) subject to a

per-address cooldown. A configurable minter address exists so the tournament system can distribute prizes.

3.3 PlayerCardNFT: dynamic ERC-1155 cards

Each card is a registered player whose career statistics (matches, runs, wickets, strike rate, economy) are stored on-chain and can be mutated by an authorized updater. Rarity sets both a fantasy multiplier and a hard supply cap:

Tier	Multiplier	Basis points	Supply cap	Effect
Common	1.0×	100	Unlimited	Base player telemetry
Uncommon	1.1×	110	5,000	Slight performance boost
Rare	1.25×	125	1,000	Enhanced scoring potential
Epic	1.5×	150	250	Visual enhancements, boosted points
Legendary	2.0×	200	25	Highest rarity, special edition art

```
function updatePlayerStats(
    uint256 cardId,
    uint32 additionalRuns,
    uint32 additionalWickets,
    uint32 newStrikeRate,
    uint32 newEconomy
) external onlyUpdaterOrOwner;
```

Listing 3.1 – Statistic mutation interface of PlayerCardNFT (from the specification).

3.4 MatchTicketNFT: anti-scalping tickets

`MatchTicketNFT.sol` is an ERC-721 contract with role-based access control (`GATEKEEPER_ROLE`, `ORGANIZER_ROLE`). Each ticket stores its face value, an on-chain match window, and a redemption record. The resale ceiling is a function of face value only (Section 4.1). Redemption is a one-way transition: `isRedeemed` flips to true, and the timestamp and gatekeeper address are logged. The contract rejects early scans, late scans, unauthorized callers, and duplicate scans.

3.5 CricketMarketplace: escrow and settlement

The marketplace takes custody of the listed asset and the buyer's CRIC in a single atomic exchange, so neither side can be left half-paid. Card sales pay a 5% creator royalty and 95% to the seller. For tickets, the marketplace calls `isPriceWithinAntiScalpingCap(tokenId, price)` on `MatchTicketNFT` before a listing is created. Settlement functions (`buyItem`, and in the tournament pool `distributePrizes` and `claimRefund`) are `nonReentrant` and follow the Checks-Effects-Interactions pattern. Listings are fixed-price orders.

3.6 CardSVGGenerator, oracle, tournaments, and polls

- **CardSVGGenerator** composes each card as a standalone SVG encoded as an RFC-2397 base64 data URI, with no external image host. Milestone badges are drawn from the stored statistics (Section 4.3).
- **CricketOracleConsumer** accepts match telemetry only from an authorized relayer or the owner. Every request has a unique 32-byte `requestId`; once `isFulfilled` is set, replaying the same ID reverts.
- **TournamentPool** escrows entry fees (50 CRIC in the demo contest), supports locking entries, and lets the organizer settle. It pays 95% of the pool to winners and 5% to the platform, and it guarantees a 100% refund if a match is rained out. Status is set to Settled before tokens move.
- **FanPolls** records one wallet-signed vote per address per poll. Vote counts are immutable once cast, and votes carry no token reward.

4 Core Mechanisms

4.1 The 15% resale ceiling

A ticket cannot be listed or resold above 115% of its face value. The ceiling is computed in basis points so the contract needs only integer arithmetic:

$$\text{MaxResalePrice} = \text{FaceValue} \times 11500 / 10000 = \text{FaceValue} \times 1.15$$

For a 100 CRIC ticket, the maximum resale price is exactly 115 CRIC. Test `SEC-01` confirms that a 116 CRIC listing reverts with the message “Marketplace: Scalping detected! Resale price exceeds 15% ceiling”. The rule is checked in both the marketplace and the ticket contract, so there is no path that skips it by going through the user interface.

4.2 Ticket lifecycle and turnstile entry

1. **Issue.** An organizer mints a ticket with a face value and a match window. The deployment seeds three tickets, one of which is live for four hours.
2. **Hold or resell.** The holder keeps the ticket or lists it on the marketplace within the 15% ceiling.
3. **Present.** The holder's wallet signs an entry pass. A deterministic vector QR code, generated in the browser from `CV-PASS-<tokenId>`, carries the payload.
4. **Scan.** A wallet with `GATEKEEPER_ROLE` scans by camera or pastes the signed payload and submits `redeemTicket(tokenId)`.
5. **Enforce.** The contract checks the role, the match window, and `!isRedeemed`, then records the redemption. A second scan reverts with “Ticket already redeemed (Double-Entry Blocked)”.

Because the QR code is only a pointer to on-chain state, a screenshot or photocopy is worthless once the real ticket has been used.

4.3 Dynamic cards and SVG badges

When statistics change, the card art changes with them, because the SVG is generated from contract state at read time. The renderer adds a badge when a threshold is met:

Badge	Condition	Statistic
CENTURION 100+	Runs ≥ 100	Batting
HALF-TON 50+	Runs ≥ 50	Batting
5-WKT HAUL / 3-WKT CRUSH	Wickets ≥ 5 / ≥ 3	Bowling
STRIKE MASTER	Strike rate ≥ 140	Batting
ECONOMY KING	Economy ≤ 6.5	Bowling

4.4 Marketplace economics

Asset	Price rule	Settlement
Player card	Fixed price set by the seller	5% creator royalty, 95% to seller, atomic swap
Match ticket	At most 115% of face value	Cap checked before listing; atomic swap

The seeded deployment contains two listings: a 125 CRIC card and a 90 CRIC ticket. A buyer must use a different wallet from the seller.

5 Fantasy Engine & Zero-Knowledge Squad Commitments

5.1 Deterministic scoring

Fantasy points are computed by a pure function in `scoringEngine.ts`, so the same match data always yields the same score:

$$\text{BasePoints} = \text{Runs}\cdot 1 + \text{Fours}\cdot 1 + \text{Sixes}\cdot 2 + \text{Wickets}\cdot 20 + \text{Catches}\cdot 8 + \text{Bonus}_{50/100}$$

$$\text{FinalPoints} = \text{BasePoints} \times \text{RarityMultiplier}$$

The milestone bonus is 10 points for 50 to 99 runs and 25 points for 100 or more. Rarity multipliers are those in Section 3.3, so a Legendary card doubles its holder's points.

5.2 Squad rules

Constraint	Rule
Squad size	Exactly 11 distinct players
Salary cap	At most 120 credits
Wicketkeepers	At least 1
Batsmen	At least 3
Bowlers	At least 3
All-rounders	At least 1

5.3 Commit-and-prove scheme

To stop copy-trading, a squad is not published. The prover holds a secret witness of eleven players and a 256-bit random salt σ, and publishes only a commitment:

$$C = \text{Keccak256}(\ \text{Sorted}(p) \parallel \sigma\ )$$

Alongside C, the public signals are the packed constraint bitmask, the tournament ID, and a timestamp. The constraints are expressed as an R1CS-style system over the BN254 field. With x_i a binary selector for player i and c_i the credit cost:

$$\Sigma_{i=1}^{N} x_i = 11, \qquad x_i(1 - x_i) = 0$$

$$1200 - \Sigma_{i=1}^{11} (10 \cdot c_i) = \Sigma_{j=0}^{10} (2^j \cdot b_j), \qquad b_j(1 - b_j) = 0$$

$$\Sigma\, x_i \cdot R_{\text{role},i} \geq \text{quota} \qquad (\text{WK} \geq 1, \text{BAT} \geq 3, \text{BOWL} \geq 3, \text{AR} \geq 1)$$

The second equation turns the inequality “total credits ≤ 120” into an equality by adding an 11-bit slack value, where credits are scaled by 10 to avoid decimals. The verifier module (key label `VK_CRIC_SQUAD_BN254_REV1`) checks BN254 base-field membership, the role quotas and the 11-player sum, the packed public signals, witness integrity, and that the commitment opens to the claimed squad. A built-in tamper test confirms that a forged proof is rejected.

> **What the chain sees.** When a player enters a tournament, the contract receives and stores the commitment hash. The `TournamentPool` contract does not itself verify the proof or open the commitment, and winners are entered by the organizer. The scheme therefore protects lineup privacy in the interface and in the commitment record, but it is not yet a trustless on-chain verification.

6 Data Pipeline & Oracle

6.1 Data sources

Source	Description
ESPN live API	Polled every 25 seconds through the `/espn-api` proxy. Updates live team scores, wickets, and player points in the interface.
Seed data	`seedData.ts` is an offline catalog of stars and fixtures, used when the live feed is unavailable.
Player catalog	`public/data/players_2000_2026.json` holds the 4,800-player database, with portraits in `public/images/players/`.
Over simulator	A deterministic six-ball engine producing runs, boundaries, extras, and dismissals with commentary text.

Live data is normalized into one internal match structure (teams, league, status, state of pre/in/post, summary text, venue, competitor scores, optional player scores) so the interface can render real and simulated matches identically.

6.2 Oracle relay

Updating a card from match data is a three-step relay. An authorized account opens a request, receiving a unique `requestId`. The relayer later calls `fulfillRequest` with the statistics, and the consumer forwards them to `PlayerCardNFT.updatePlayerStats`. Only addresses passing `onlyOracleOrOwner` can fulfil a request, and a fulfilled request cannot be replayed. The contract is Chainlink-compatible in shape, but no Chainlink Functions subscription is configured: today the relayer is an owner-controlled account, and the demo fixture windows and simulated points are generated for practice rather than taken from verified results.

7 Frontend & User Experience

7.1 Application structure

Screen	Component	Purpose
The Pavilion	`PavilionTab.tsx`	Browse the athlete catalog and mint cards with CRIC.
My Collection	`CollectionTab.tsx`	Portfolio value, tiers owned, card inspection, one-click listing.
Auction Room	`MarketplaceTab.tsx`	Browse and buy listings; list owned cards and tickets.
Matchday XI	`FantasyTab.tsx`	Build a squad, generate and test a squad proof.
Stadium Gate	`TicketsTab.tsx`	Resale-cap auditor, QR pass, camera/paste turnstile scanner.
Polls	`PollsTab.tsx`	Cast wallet-signed fan votes.
Tournaments	`TournamentsTab.tsx`	Join contests, view upcoming, live, and settled pools.

7.2 State and wallet handling

`CricketContext.tsx` is the single source of truth for wallet connection, network, balances, contract handles, match synchronization, and ticket state. `TransactionConfirmationModal.tsx` presents

each write transaction before the wallet prompt. On the local chain only, the header offers “Get 20 demo ETH” and “Claim 50,000 CRIC”. The ETH helper is enabled only for chain ID 31337 from localhost; every feature transaction still needs the wallet's signature.

7.3 Design system and offline support

The “Dark Turf” design system in `public/style.css` takes its palette from a floodlit stadium: turf green for primary actions and verified states, cricket white on dark slate for contrast, boundary gold for legendary and milestone elements, and ball red for wickets and violations. Numeric data such as hashes and live scores use a monospaced face. The service worker (`sw.js`) and manifest (`manifest.json`) make the app installable and let a gatekeeper phone keep working with a poor connection.

8 Testing, Security & Gas Analysis

8.1 Automated tests

The repository contains 47 passing Hardhat tests across four suites, run with `npx hardhat test` against a local EVM. The count is a record of tested behaviors, not a coverage percentage.

Suite	Scope
`CricketVerse.test.js`	Core token, cards, tickets, and marketplace behavior.
`Phase2_Ecosystem.test.js`	SVG generator, oracle relayer, tournament escrow.
`Phase3_Security_Gas.test.js`	Security checks SEC-01 to SEC-07 and gas benchmarks.
`Phase4_OnchainCommunity.test.js`	Fan polls and scheduled ticket windows.

8.2 Threat model

Threat	Status	Defence
Reentrancy (SWC-107)	Mitigated	OpenZeppelin `ReentrancyGuard` on marketplace and pool; payout functions are `nonReentrant`; state set to Settled before token transfers.
Scalping / price gouging	Verified	On-chain 115% ceiling; test SEC-01 shows a 116 CRIC listing reverting.
Double entry / QR replay	Mitigated	`isRedeemed` flips permanently on first scan; re-scan reverts.
Oracle spoofing / replay	Mitigated	`onlyOracleOrOwner` access and one-shot `requestId` fulfilment.
Privilege escalation	Verified	AccessControl roles and `Ownable`; unauthorized calls revert with `AccessControlUnauthorizedAccount`.
Unauthorized minting	Mitigated	Minter role plus hard supply caps on rare tiers.
Front-running (MEV)	Reduced	Fixed-price atomic fills verified at execution; squads hidden behind commitments.

8.3 Gas profile

Measurements were taken on the Cancun target with `viaIR` and 200 optimizer runs. Costs assume 30 Gwei on an L2 costs less in dollar terms, mostly because of the price of the native token. Ticket issuance and marketplace listing are the heaviest operations because they write several storage slots.

Operation	Gas	Ethereum L1 (30 Gwei)
CRIC transfer	34,432	0.00103 ETH ($3.10)
PlayerCardNFT mint	96,812	0.00290 ETH ($8.71)
MatchTicketNFT issue	284,795	0.00854 ETH ($25.63)

Turnstile check-in	80,299	0.00241 ETH ($7.23)
Marketplace list ticket	259,060	0.00777 ETH ($23.32)
Marketplace atomic buy	151,243	0.00454 ETH ($13.61)
Oracle fulfil stats	71,180	0.00214 ETH ($6.41)
Tournament create escrow	212,978	0.00639 ETH ($19.17)
Tournament join escrow	206,906	0.00621 ETH ($18.62)
Tournament 95/5 settlement	107,027	0.00321 ETH ($9.63)

9 Deployment & Operation

9.1 Setup procedure

Prerequisites are Node.js 18 or later, npm 9 or later, Git, and a MetaMask-compatible browser wallet. Two terminals are needed:

```
npm install                  # dependencies
cp .env.example .env         # local test settings
npx hardhat compile          # viaIR, 200 runs
npm run chain:node           # terminal 1: Cancun node on 127.0.0.1:8545
npm run chain:deploy         # terminal 2: deploy all 8 contracts and seed demo data
npm run dev                  # Vite dev server on http://localhost:3000
node scripts/reset_user_wallet.js   # optional: restart a wallet from 0 CRIC
```

Listing 9.1 – Commands to run the local demo.

In MetaMask, add a network with RPC `http://127.0.0.1:8545`, chain ID 31337, and currency ETH, then import a pre-funded local test account or use the header buttons to fund a fresh wallet. Restarting the Hardhat node resets the chain, so `chain:deploy` must be run again so that `public/contracts-config.json` matches the new addresses.

9.2 Seeded demo state

A fresh deployment seeds three registered cards, three tickets, two marketplace listings, two polls, and upcoming, live, and settled tournaments. This lets every screen be exercised immediately:

Feature	Walkthrough
Collect	Claim CRIC from the faucet, then mint a registered card in the Pavilion.
Trade	Buy the 125 CRIC card or 90 CRIC ticket from a second wallet, then relist an owned item.
Compete	Build an XI, approve the 50 CRIC entry, and join the open contest; the organizer locks and settles.
Vote	Choose a poll option; each wallet can vote once per poll.
Enter the stadium	The holder signs an entry pass; a gatekeeper wallet verifies it. Early, late, unauthorized, and duplicate scans are rejected.

> **Key safety.** The keys in `.env.example` and the MetaMask guide are the well-known Hardhat test accounts. They must never be used on a public network or funded with real assets.

10 Limitations & Future Work

10.1 Design intent versus current state

The documentation describes a larger vision in places than the code currently delivers. The table separates the two so the project is represented accurately.

Area	Design intent	Current state
Oracle	Chainlink-style live sports data	Owner-authorized relayer; no Chainlink Functions subscription; demo fixtures and points are generated.
Squad proof	Groth16 verification on BN254	Proof and commitment generated in the browser; the pool stores the hash and does not verify or open it.
Tournament result	Automated, trust-minimized settlement	Organizer reviews and enters winners.
Marketplace	English auction with 10-minute anti-sniping	Fixed-price escrow orders (as stated in the setup guide and security review).
Governance	Quadratic voting, $W = \lfloor \sqrt{V} \rfloor$, 72-hour windows	One wallet-signed vote per address per poll; votes only record a choice.
Network	Polygon Amoy L2	Local Hardhat only; no funded deployer key, public deployment, or HTTPS host.
Assurance	Production-grade security	47 local tests; no independent audit or formal verification.

10.2 Future work

- **On-chain proof verification.** Generate a verifier contract for the squad circuit using the BN254 precompiles (`ecAdd`, `ecMul`, `ecPairing`), add a reveal phase that opens each commitment after lock, and reject entries whose proof fails.
- **Trustworthy results.** Replace the owner relayer with Chainlink Functions or a multi-signer oracle, and settle tournaments automatically from verified scorecards.
- **Timed auctions and richer governance.** Implement soft-close English auctions and the quadratic-voting contract described in the design.
- **Public testnet launch.** Deploy to Polygon Amoy, host over HTTPS, and measure real transaction costs.
- **Assurance.** Add fuzz and invariant tests, measure coverage, and commission an independent audit before any use with real value.
- **Account abstraction (ERC-4337).** Gasless pass purchases through paymasters, session keys for one-tap gate entry, and social recovery for fans new to wallets.

11 Conclusion

CricketVerse shows that the main complaints about digital cricket platforms can be addressed with contract-level rules: a ticket that cannot be resold above 115% of face value and cannot be used twice, a collectible whose artwork follows the player's statistics, escrowed marketplace and prize funds with atomic settlement and refunds, and a fantasy squad that stays private until it is needed. The eight contracts, 47 passing tests, seeded demo, and complete React interface make the design demonstrable end to end on a local chain.

Its honest status is a well-structured course prototype. The oracle, the on-chain verification of squads, automated settlement, and a public-network deployment are the next steps that would turn the demonstrated mechanisms into a trust-minimized product.